

DEMAND FORECASTING MODEL USING PYSPARK

Technology Stack Apache PySpark MinIO Docker Python	Dataset Size 76,000 Records 16 Features Multi-Region
Models Used Linear Regression Random Forest GBT ARIMA	Storage MinIO (S3-Compatible) Parquet Distributed

Name	ID
Ziad Elkhateb	22010100
Mustafa Elshimy	22010395
Abdelrahman Ahmed	22011620
Nahed Refay	22010280

1. Project Overview

1.1 Project Description

This project implements a comprehensive Demand Forecasting pipeline for e-commerce products using Apache PySpark. The primary goal is to improve supply chain efficiency, reduce stockouts, and optimize inventory management by leveraging historical sales data and advanced machine learning analytics in a distributed computing environment.

The solution integrates data ingestion, distributed storage, exploratory data analysis, feature engineering, model training, performance optimization, and MLOps deployment into a cohesive end-to-end pipeline — fully orchestrated with PySpark and backed by a MinIO object storage cluster.

1.2 Project Objectives

- Predict future product demand accurately using machine learning models
- Identify seasonal trends, purchasing patterns, and demand drivers
- Optimize inventory and supply chain decisions based on data-driven forecasts
- Improve business efficiency and reduce costs through better stock management
- Build a scalable, distributed pipeline capable of handling large volumes of transactional data

1.3 Technology Stack

Component	Technology / Detail
Distributed Computing	Apache PySpark (local[*] / cluster mode)
Object Storage	MinIO (S3-compatible) via Docker Compose
File Format	Parquet (columnar, partitioned by Region & Category)
ML Framework	PySpark MLlib, Statsmodels (ARIMA)
Visualization	Matplotlib, Seaborn
Containerization	Docker Compose (MinIO service)
Language	Python 3.x, Jupyter Notebook

2. Dataset Description

2.1 Dataset Overview

The dataset contains 76,000 records of e-commerce sales transactions spanning multiple stores, products, regions, and time periods. Each record captures a comprehensive snapshot of demand-related attributes on a given date.

2.2 Dataset Schema

Column Name	Data Type	Description
Date	Date	Transaction date (YYYY-MM-DD)
Store ID	String	Unique identifier for each store
Product ID	String	Unique identifier for each product
Category	String	Product category (Electronics, Groceries, Clothing, Furniture)
Region	String	Geographical region (North, South, East, West)
Inventory Level	Integer	Current stock level at time of record (0–2267)
Units Sold	Integer	Actual units sold on the date (0–426)
Units Ordered	Integer	Units ordered for restocking (0–1616)
Price	Double	Product selling price in USD (4.74–228.03)
Discount	Integer	Discount percentage applied (0–25%)
Weather Condition	String	Prevailing weather (Snowy, Rainy, Sunny, Cloudy)
Promotion	Binary	Whether a promotion was active (1 = Yes, 0 = No)
Competitor Pricing	Double	Competitor product price in USD (4.29–261.22)
Seasonality	String	Seasonal label (Winter, Spring, Summer, Fall)
Epidemic	Binary	Epidemic indicator flag (1 = Yes, 0 = No; 20% prevalence)
Demand	Integer	TARGET: Actual product demand (4–430, avg 104)

2.3 Statistical Summary

Feature	Mean	Std Dev	Min	Max	Median
Inventory Level	301.1	226.5	0	2,267	227
Units Sold	88.8	44.0	0	426	84
Price	\$67.73	\$39.38	\$4.74	\$228.03	\$64.50
Demand (Target)	104.3	47.0	4	430	100

3. Project Roles & Responsibilities

The project follows a distributed team structure where five engineering roles contribute distinct expertise to deliver an end-to-end demand forecasting solution.

3.1 Data Engineer — Ingestion & Storage

Data Engineer

- Read and load raw CSV data using PySpark with inferred schema
- Cast date fields from String to DateType for temporal analysis
- Check and validate data quality: null values, duplicate rows
- Extract temporal features: Year, Month, Day of Week
- Store processed data in distributed MinIO storage as Parquet

3.2 Data Analyst — Exploration & Insights

Data Analyst

- Perform comprehensive Exploratory Data Analysis (EDA)
- Analyze demand trends by category, region, and seasonality
- Generate correlation analysis across numeric features
- Build visualizations: bar charts, heatmaps, histograms, scatter plots
- Identify business insights and demand drivers

3.3 Machine Learning Engineer — Model Development

ML Engineer

- Build time-aware features: lag-1, lag-7, rolling 7-day average demand
- Engineer derived features: inventory gap, price gap, sell-through ratio
- Perform time-based train/test split (80/20) to prevent data leakage
- Train and evaluate Linear Regression, Random Forest, Gradient Boosting
- Implement ARIMA(7,1,2) as a classical time-series baseline

3.4 Big Data Engineer — Optimization & Performance

Big Data Engineer

- Optimize Spark jobs through repartitioning (8 partitions by Region & Category)
- Persist engineered dataset to avoid repeated recomputation
- Write optimized feature store as partitioned Parquet to MinIO
- Configure MinIO bucket replication using mc CLI for fault tolerance
- Analyze execution plans with Spark explain() for query optimization

3.5 MLOps Engineer — Deployment & Monitoring

MLOps Engineer

- Save trained preprocessing pipeline and best model to MinIO
- Implement batch inference function for production use
- Monitor forecast accuracy daily using MAE and MAPE metrics
- Download saved models locally using boto3 for offline deployment
- Automate data pipeline steps and model evaluation workflows

4. Data Pipeline & Processing

4.1 Data Ingestion

The data ingestion phase begins with initializing a `SparkSession` configured to run in local mode with all available CPU cores. The session is also configured with the S3A connector for communication with MinIO, making Spark capable of reading and writing data directly to the object store using the `s3a://` protocol.

Spark Configuration Highlights:

- `spark.sql.shuffle.partitions = 8` (tuned for local execution)
- `spark.hadoop.fs.s3a.endpoint = http://127.0.0.1:9000`
- Credentials: `minioadmin / minioadmin` (development environment)
- Fast upload enabled with `ByteBuffer` for improved MinIO write throughput

4.2 Data Cleaning & Preprocessing

- Date Conversion: String-type Date column converted to `DateTime` using `to_date()` with format `yyyy-MM-dd`
- Null Value Audit: Applied per-column null checks with NaN detection for floating-point columns — no missing values found
- Duplicate Detection: Compared total row count vs. distinct row count — no duplicate records identified
- Feature Extraction: Year, Month, and DayOfWeek extracted from Date for temporal modeling
- Column Pruning: Date, Store ID, and Product ID dropped from the analytical dataset before ML processing

4.3 Distributed Storage — MinIO

The cleaned dataset is persisted to MinIO using the `s3a://` connector. Three buckets are used to organise data across the pipeline lifecycle:

Bucket	Purpose	Format
demand-lake	Primary data lake: raw, features, models, optimized store	Parquet (partitioned)
cleaned-data	Preprocessed dataset for analytical consumption	Parquet (partitioned by Region, Seasonality)
backup-data	Replication target for fault tolerance	Parquet (replicated via mc)

The docker-compose.yml file included in the project provisions the MinIO container with API port 9000 and a web console at port 9001. Bucket replication is configured using the MinIO Client (mc) command-line tool.

5. Exploratory Data Analysis & Visualizations

5.1 Average Demand by Product Category — Bar Chart

Insight: Groceries show consistently high demand volumes driven by daily consumption patterns, while Electronics display high-value but lower-frequency purchasing behaviour

5.2 Distribution of Demand — Histogram

Insight: The demand distribution is right-skewed with a mean of approximately 104 units. The long right tail indicates occasional high-demand events that require special inventory treatment.

5.3 Boxplot — Key Feature Distributions

Boxplots reveal the spread, median, and outlier structure of the four most important numeric features in the dataset.

5.4 Correlation Heatmap

Insight: Units Sold shows the strongest positive correlation with Demand, confirming it as a primary predictor. Inventory Level has a modest negative correlation, indicating that lower stock tends to coincide with lower recorded demand — a potential stockout signal.

5.5 Seasonality Distribution — Pie Chart

Insight: Winter records are the most frequent in the dataset. All four seasons are well-represented, ensuring the model can learn seasonal demand patterns without class imbalance bias.

5.6 Units Sold vs. Demand — Scatter Plot

Insight: A strong positive linear relationship exists between Units Sold and Demand (slope approximately 1). Points clustered around the regression line confirm that this feature is highly predictive of demand.

5.7 Regional Sales Analysis — Stacked Bar Chart

The stacked horizontal bar chart breaks down total units sold by both region and product category, revealing geographic and categorical sales patterns.

Key Insights:

- North dominates total sales volume, nearly doubling individual output of other regions
- Groceries is the primary sales driver across all regions
- West shows disproportionately high Furniture sales compared to other regions
- South has a strong Clothing contribution relative to its total volume

5.8 Demand vs. Units Sold — Time Trend

Insight: Total Demand consistently exceeds Units Sold throughout the observation period. This persistent gap indicates chronic stockout conditions and represents the core business problem this project addresses. Seasonal spikes are visible, particularly in the winter months.

6. Feature Engineering

6.1 Temporal Features

Year, Month, DayOfWeek — extracted from the Date column to capture seasonal and weekly demand cycles

6.2 Lag Features (Window Functions)

- lag_1_demand — demand from the previous day per product, capturing day-over-day momentum
- lag_7_demand — demand from 7 days prior per product, capturing weekly periodicity
- rolling_7_avg_demand — 7-day backward rolling average demand per product, smoothing short-term noise

6.3 Derived Business Features

- inventory_gap = Inventory Level - Demand, measuring whether stock can fulfill demand
- price_gap = Price - Competitor Pricing, measuring competitive pricing position
- sell_through_ratio = Units Sold / Inventory Level, measuring how quickly inventory is depleted
- order_fill_ratio = Units Sold / Units Ordered, measuring supply chain order fulfillment efficiency

6.4 Categorical Encoding

Categorical columns (Category, Region, Weather Condition, Seasonality) were encoded using a two-step pipeline of StringIndexer followed by OneHotEncoder within a PySpark ML Pipeline. All numeric and encoded features were then assembled into a single features vector using VectorAssembler.

6.5 Train / Test Split Strategy

Time-Based Split (80% / 20%):

- Training set: all dates before the 80th percentile date cut-off
- Test set: all dates from the cut-off onward
- Rationale: prevents future data leakage — random splits are not valid for time-series forecasting

7. Machine Learning Models

7.1 Models Overview

Four forecasting models were trained and evaluated: three distributed Spark MLlib regression models and one classical univariate time-series model.

7.2 Model Configurations

Model	Key Hyperparameters	Notes
Linear Regression	maxIter=50, regParam=0.1, elasticNetParam=0.2	ElasticNet regularisation; trained on full training set
Random Forest	numTrees=30, maxDepth=6, subsamplingRate=0.8	35% sampled training data to reduce memory pressure
Gradient Boosting	maxIter=20, maxDepth=4, stepSize=0.1	GBTRegressor; 35% sampled training data
ARIMA(7,1,2)	p=7, d=1, q=2	Univariate model on daily aggregated demand only

7.3 Model Performance Results

Model	RMSE	MAE	R ²	Rank
Linear Regression	13.515941	10.193616	0.906010	1st (Best)
Gradient Boosting	16.011815	12.611173	0.868093	2nd
Random Forest	17.065201	13.374107	0.850166	3rd
ARIMA(7,1,2)	2084.164979	1415.414081	-0.441525	4th

- Linear Regression delivers the best error profile and highest **R²** indicating strong explanatory power for the engineered feature set.
- Tree models perform well but trail the linear baseline, likely due to sampling and local resource limits.
- ARIMA underperforms with a negative **R²** reinforcing the value of multivariate features beyond univariate temporal history.

8. Big Data (Optimization & Performance)

8.1 Big Data Optimization

- Shuffle partitions set to 8 for local stability
- Repartition on Region and Category to align with aggregation patterns
- Persisted engineered dataset to avoid recomputation
- Partitioned parquet for pruning and faster downstream reads
- Tree models trained on a sample to avoid local JVM pressure
- Partitions before optimization = 3, after repartition = 8

8.2 Storage and Replication Strategy

Data Lake Layout:

- Cleaned dataset: s3a://cleaned-data/cleaned-demand/
- Optimized feature store: s3a://demand-lake/optimized-feature-store/
- Models and preprocessing pipeline: s3a://demand-lake/models/

output:

- Cleaned dataset saved to s3a://cleaned-data/cleaned-demand/ (current Spark partitions: 2)
- Optimized feature store written to s3a://demand-lake/optimized-feature-store/
- Preprocessing pipeline saved to s3a://demand-lake/models/preprocessing-pipeline/
- Best model saved to s3a://demand-lake/models/linear_regression/

8.3 Replication and Fault Tolerance

- MinIO replication configured via mc with versioning enabled
- Source bucket (cleaned-data) replicates to backup-data
- Object storage redundancy ensures recovery from accidental deletions or corruption

9. MLOps — Deployment & Monitoring

9.1 Model Serialisation

After training, both the preprocessing Pipeline model and the best forecasting model (Gradient Boosting Regressor) were serialised and saved to the MinIO demand-lake bucket under the `models/` prefix using Spark's native `.write().overwrite().save()` interface.

9.2 Batch Inference

A `run_batch_inference()` function encapsulates the end-to-end prediction workflow: it applies the saved preprocessing pipeline to raw input data and then runs the trained model's `transform()` method to generate predictions. A simplified `predict_demand()` wrapper exposes the Linear Regression model for single-sample prediction.

9.3 Performance Monitoring

Daily model monitoring is implemented by computing two error metrics per day from the test set predictions:

- Mean Absolute Error (MAE) — average absolute difference between predicted and actual demand
- Mean Absolute Percentage Error (MAPE) — normalised error expressing forecast deviation as a percentage

A daily MAE trend chart is generated to surface model drift or systematic bias over time. If daily MAE exceeds a defined threshold, the model should be retrained on more recent data.

9.4 Local Model Retrieval

Because saving Spark models directly to Windows local paths can trigger Hadoop DLL errors, the project uses `boto3` to download the serialised models from MinIO to a local `artifacts/models/` directory. The `download_s3_directory()` utility recursively downloads all objects under a given S3 prefix, preserving the folder structure required by PySpark's model loader.